

Executive Summary of Project 3

-Ananya Asthana

Assignment Overview

The purpose of this assignment was to develop an understanding of how the two-phase commit (2PC) protocol works in distributed systems. The objective was to implement a system where multiple server replicas are created, and a client can connect to any of them and perform string operations such as PUT, GET, and DELETE. The operations should be propagated across all servers, which maintain their own key-value stores. Thus, the data store should be consistent across all replicas, even in the presence of failures. Before an operation is carried out, the coordinator needs to send a prepare request to all servers. Once all servers confirm readiness, the coordinator initiates the commit phase across all replicas. If any server cannot commit, the coordinator triggers a rollback to maintain consistency across the system. The system was designed to handle concurrent client requests using multithreading, ensuring that operations across all servers remain consistent and atomic. The implementation required setting up RMI-based communication for client-server interactions, configuring the coordinator and participant structure, and ensuring that registry conflicts and multithreaded request handling were properly managed. The assignment involved coordinating multiple servers, managing state consistency, and handling failure scenarios, which provided insight into real-world distributed system challenges.

Technical Impression

My experience was productive, since this assignment helped me understand how multi-server environments work. It helped me develop a clear understanding of how a coordinator facilitates 2PC and maintains consistency amongst all servers, as well as ensures that all servers are ready before it moves to the next step. Setting up the RMI-based communication and managing registry conflicts between the coordinator and participants was initially challenging. Handling parallel client requests required careful multithreading, which was managed using a `ConcurrentHashMap` for thread safety. Implementing the prepare and commit phases taught me the importance of state management and failure handling. It was initially a little difficult to structure how my client would contact the server, whether it should directly contact the `KeyValueStoreOperations` functions, but later I figured out the best approach was for the client to contact the participant, which in turn contacts the coordinator to complete the 2PC process. Understanding this flow was an educative experience, especially when dealing with failure scenarios where rollback had to be initiated correctly if any participant failed to commit. Overall, it provided valuable insights into real-world challenges faced in building scalable and fault-tolerant distributed systems.